
Token-Level Subspace Routing: A New Paradigm for Transformer Hidden Representations

Anonymous Authors¹

Abstract

Transformers allocate the same d -dimensional hidden representation to every token, regardless of its semantic complexity. This uniform capacity allocation is a long-standing architectural assumption that conflicts with empirical evidence: token activations and hidden features concentrate on highly structured, low-dimensional subspaces. We identify *token-level capacity allocation* as a fundamental but previously unaddressed modeling problem.

We introduce *Token-Level Subspace Routing* (TLSR), a new paradigm in which each token dynamically activates only a small subset of learned hidden subspaces. TLSR factorizes the hidden dimension, learns a token-dependent routing mechanism, and induces structured block-sparse computations for both FFN and attention projections—while keeping the Transformer architecture unchanged. Unlike MoE or activation sparsity, TLSR redistributes existing capacity at a finer granularity without introducing experts or unstructured sparsity.

Empirically and theoretically, we show that token representations inhabit multiple low-dimensional manifolds, and that TLSR aligns computation with this geometry while enabling efficient block-sparse GPU kernels. Across large language model benchmarks, TLSR matches or improves accuracy while reducing FLOPs and wall-clock latency promising and general direction for scaling efficient Transformers.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Modern Transformer-based large language models (LLMs) allocate the same d -dimensional hidden representation to every token at every layer. This architectural choice—an implicit “uniform capacity allocation assumption”—has remained unchanged since the original Transformer design, despite the fact that different tokens vary dramatically in semantic complexity, contextual dependence, and intrinsic dimensionality. Common function words, punctuation, and structural markers rarely require the same representational capacity as content-bearing tokens, yet all tokens are processed with identical full-dimensional linear projections.

Recent analyses provide growing evidence that Transformers do not use their hidden dimensions uniformly. A substantial portion of neurons remain inactive across many contexts (“lazy neurons”), revealing strong activation sparsity and highly uneven feature utilization (Li et al., 2023; Liu et al., 2024a; Zhang et al., 2024). At the same time, representation geometry studies show that embedding and hidden layers exhibit structured subspaces and multi-manifold organization, rather than a homogeneous vector space (Baevski & Auli, 2019; Mehta et al., 2020; Mickus et al., 2022). Together, these findings suggest a fundamental mismatch: *the Transformer allocates representational capacity uniformly, even though token representations naturally concentrate in specialized subspaces.*

Prior attempts to exploit this mismatch focus on different levels of granularity. Mixture-of-Experts (MoE) architectures route tokens to a small number of large experts, activation sparsity methods drop neurons dynamically, and structured sparsity approaches prune attention or FFN blocks (Farina et al., 2024; Yu et al., 2023; Shen et al., 2024c). However, these techniques either rely on unstructured sparsity (difficult to accelerate on GPUs), duplicate parameters across experts, or alter the Transformer architecture. Crucially, none of them address the core representational question: *how should a Transformer allocate its existing hidden dimensions across tokens in a geometry-aware, fine-grained, and computationally efficient way?*

In this paper, we formulate **token-level capacity allocation** as a first-class modeling problem. Instead of treating the

hidden space $\mathbb{R}^d$ as a monolithic object, we propose decomposing it into multiple learned subspaces and routing each token to only a small subset at each layer. This perspective views the hidden space as a structured resource that can be redistributed across tokens based on their representational needs, aligning model computation with the intrinsic geometry of token manifolds.

We introduce *Token-Level Subspace Routing* (TLSR), a new paradigm for Transformer hidden representations. TLSR factorizes the hidden dimension into M subspaces and equips each token with a gating function that selects $k \ll M$ subspaces at each layer. Both FFN and attention projection matrices are partitioned accordingly, so that only the routed blocks are computed for each token. Unlike MoE, TLSR does not introduce experts or additional parameters; unlike sparsity methods, it produces structured block-sparse patterns that are natively GPU-friendly. TLSR leaves the Transformer architecture intact and can be applied to any existing model without altering its block structure.

Beyond efficiency, TLSR operationalizes a representational geometry viewpoint: if token representations inhabit multiple low-dimensional submanifolds, then the model should explicitly route tokens to the subspaces aligned with these manifolds. We provide empirical evidence supporting this view, showing that TLSR-learned subspaces capture meaningful linguistic functions and that routing patterns correlate with token complexity. We further give theoretical insights connecting subspace routing to expressive capacity and approximation behavior under compute constraints, complementing recent work on subspace decomposition and representation disentanglement (Huang & Hahn, 2025; Zhou, 2025).

Our contributions are as follows:

- We identify **token-level capacity allocation** as a fundamental but previously unaddressed modeling problem in Transformers, grounded in empirical activation sparsity and multi-submanifold representation geometry.
- We propose **Token-Level Subspace Routing** (TLSR), the first framework that dynamically routes tokens among learned hidden subspaces, inducing structured block-sparse computation while preserving the standard Transformer architecture.
- We develop a complete systems-ready design that supports efficient block-sparse FFN and attention projections, leveraging GPU-friendly sparsity layouts inspired by recent advances in hardware-aligned sparse operators (Yu et al., 2023; Shen et al., 2024c).
- We provide empirical and theoretical evidence that TLSR improves compute efficiency while maintaining or improving accuracy across LLM benchmarks, and

that its learned subspaces correspond to interpretable structure in token representations.

Together, these results establish token-level subspace routing as a promising and general paradigm for scaling efficient, geometry-aware Transformers.

2. Background and Motivation

2.1. Transformers and Hidden Representations

Transformers map each token at each layer to a d -dimensional hidden vector and apply the same set of linear projections and non-linearities to all tokens. This uniform design has become the default for large language models: the hidden space $\mathbb{R}^d$ is treated as a single monolithic resource that every token fully occupies, independent of its semantic role or intrinsic complexity. In practice, however, recent empirical work has shown that Transformer hidden states are far from dense and homogeneous. Studies on activation sparsity observe that many neurons are rarely or only very selectively active across tokens and layers, revealing strong structure in how capacity is actually used (Li et al., 2023; Liu et al., 2024a; Zhang et al., 2024; Farina et al., 2024). At the same time, work on embedding and hidden-space parameterizations demonstrates that word representations can be effectively factorized into multiple components, rather than relying on a single dense vector (Baeviski & Auli, 2019; Mehta et al., 2020).

Beyond sparsity, representation-geometry analyses show that Transformer embedding and hidden spaces exhibit clear subspace and multi-manifold structure. For example, Mickus et al. (2022) dissect Transformer embedding spaces and show that different linguistic functions align with different directions or regions, while recent unsupervised decomposition methods explicitly recover interpretable subspaces in model representations (Huang & Hahn, 2025; Zhou, 2025). These findings together suggest that the hidden space already organizes itself into meaningful substructures, even though the architecture continues to allocate a full d -dimensional vector to every token.

2.2. Limitations of Uniform Hidden Capacity

The uniform hidden-capacity assumption is limiting from both a representational and a computational perspective. On the representational side, different token types—such as function words, punctuation, domain-specific entities, or structured markers—can have very different intrinsic complexity and contextual dependence. Forcing all of them to occupy the same full d -dimensional space ignores the observed heterogeneity in activation and subspace usage. Empirically, activation sparsity studies show that a substantial fraction of neurons remain effectively “lazy” for many

tokens (Li et al., 2023; Liu et al., 2024a), while subspace decomposition methods indicate that only a subset of directions is relevant for particular functions or phenomena (Mickus et al., 2022; Huang & Hahn, 2025; Zhou, 2025).

On the computational side, uniform capacity means that every token incurs the full cost of all projection matrices in attention and FFN layers, even when most of the hidden dimensions contribute little to its specific representation. A large body of work therefore aims to reduce this cost through sparsity and structured pruning, including activation sparsity, semi-structured sparsity, and hardware-aware patterns that can be accelerated on modern GPUs (Farina et al., 2024; Yu et al., 2023; Fang et al., 2024; Zheng et al., 2024; Shen et al., 2024a; Liu et al., 2025a; Shen et al., 2024b; Liu et al., 2024b; Shen et al., 2024c; Yuan et al., 2025; Li & Xu, 2025; Xu et al., 2025; Xiao et al., 2025). These approaches operate at the level of neurons, blocks, or attention patterns, and have demonstrated that substantial computation can be removed without catastrophic loss of accuracy. However, they typically do not change the underlying assumption that each token is represented as a full d -dimensional vector, nor do they provide a mechanism for redistributing hidden capacity across tokens based on their representational needs.

2.3. Problem Formulation: Token-Level Capacity Allocation

The empirical picture that emerges is that Transformer hidden spaces are structured and heterogeneous, yet the architecture still allocates capacity uniformly. This motivates us to treat *token-level capacity allocation* as an explicit modeling problem. Instead of viewing the hidden space $\mathbb{R}^d$ as a single undifferentiated vector space, we consider decomposing it into multiple learned subspaces and allowing each token to selectively access only a subset of them at each layer. From this perspective, the central question becomes:

Given a fixed hidden dimension d , how can we learn a token-dependent allocation of hidden subspaces that (i) preserves or improves model quality, (ii) significantly reduces computation, and (iii) remains compatible with standard Transformer blocks and hardware-efficient sparse kernels?

Existing work on sparsity, structured pruning, and representation subspaces (Farina et al., 2024; Mickus et al., 2022; Huang & Hahn, 2025; Zhou, 2025; Yu et al., 2023; Shen et al., 2024c) provides important pieces of this picture, but does not offer a mechanism for routing *within* the hidden representation of a Transformer layer. Our goal in this paper is to fill this gap by introducing a framework that performs token-dependent subspace selection while inducing GPU-friendly block-sparse computation.

3. Token-Level Subspace Routing

This section introduces Token-Level Subspace Routing (TLSR), a new paradigm for Transformer hidden representations in which each token selectively activates only a subset of learned subspaces. TLSR restructures the hidden dimension into multiple blocks and learns a routing function that determines which subspaces are used for each token. Unlike MoE-style routing that assigns whole feed-forward experts, TLSR operates *inside the hidden dimension itself*, enabling fine-grained adaptive capacity allocation without modifying the surrounding Transformer block. We provide a complete description of the factorization, routing, integration, and training components, and conclude with an expressivity analysis showing that TLSR retains the representational capacity of dense Transformers.

3.1. Overview of the TLSR Framework

TLSR starts from the observation that Transformer hidden vectors encode heterogeneous information: some tokens (e.g., punctuation, stopwords) require only shallow transformations, while others (e.g., entities, long-range syntactic anchors) require substantially richer processing. Dense Transformers allocate the same d -dimensional space and compute the same matrix multiplications for every token, which leads to uniform capacity assignment and redundant computation.

TLSR introduces a three-stage mechanism:

1. **Hidden-space factorization:** The d -dimensional hidden vector is decomposed into a direct sum of m subspaces $\mathbb{R}^d = \bigoplus_{i=1}^m \mathbb{R}^{d_i}$, implemented either by fixed channel grouping or a learned orthogonal transform.
2. **Token-to-subspace routing:** For each token representation h_t , a routing function selects a top- k subset of subspaces to activate. Only these blocks participate in downstream linear projections.
3. **Structured block-sparse computation:** Both FFN layers and Q/K/V projections are implemented as block-sparse matrices whose block activation patterns depend on token routing. This yields real speedups because blocks are fixed-size and GPU-friendly.

TLSR obeys three design principles: (i) maintain full compatibility with standard Transformers; (ii) enforce structured factorization that preserves hardware alignment; (iii) ensure routing is token-dependent but block-regular, enabling stable optimization and efficient kernels.

3.2. Subspace Factorization of the Hidden Space

Let $h_t \in \mathbb{R}^d$ be the hidden state for token t . TLSR factorizes the hidden dimension into m subspaces:

$$\mathbb{R}^d = \mathbb{R}^{d_1} \oplus \dots \oplus \mathbb{R}^{d_m}. \quad (1)$$

We denote the block decomposition by:

$$h_t = [h_t^{(1)}, \dots, h_t^{(m)}], \quad h_t^{(i)} \in \mathbb{R}^{d_i}. \quad (2)$$

Two forms of factorization are supported:

(A) Fixed channel grouping. The simplest method splits the d dimensions into contiguous groups. This ensures perfect alignment with block-sparse kernels such as CUTLASS.

(B) Learned orthogonal transform. We learn a matrix $U \in \mathbb{R}^{d \times d}$ with a near-orthogonality regularizer, compute $\tilde{h}_t = U h_t$, and then partition $\tilde{h}_t$ into blocks. This allows subspaces to capture functional or semantic modules (Mickus et al., 2022; Huang & Hahn, 2025).

Unlike LoRA or adapters, which add low-rank deltas to weights, TLSR restructures the *main representation space*. It neither adds parameters unnecessary for dense paths nor restricts expressivity to a single low-rank decomposition.

3.3. Token-to-Subspace Gating Mechanism

TLSR routes each token to a subset of subspaces. Given hidden state h_t , we compute gating logits:

$$g_t = W_g h_t \in \mathbb{R}^m. \quad (3)$$

A softmax with temperature τ produces routing probabilities:

$$p_t = \text{softmax}(g_t / \tau). \quad (4)$$

Top- k selection yields the active subspaces:

$$S(t) = \text{Top}_k(p_t). \quad (5)$$

For each subspace $i \in S(t)$, the projector $W^{(i)}$ is used; unselected blocks are skipped, giving structured sparsity.

Preventing collapse. Training uses two regularizers:

- **Load balancing** encourages uniform usage:

$$\mathcal{L}_{lb} = m \sum_{i=1}^m \left(\frac{1}{T} \sum_t p_{t,i} \right)^2.$$

- **Gating entropy** prevents overly confident early routing:

$$\mathcal{L}_{ent} = - \sum_t H(p_t).$$

These are standard in sparsity and MoE literature (Li et al., 2023; Liu et al., 2025b), adapted to fine-grained subspaces.

3.4. Integration with Transformer Blocks

TLSR minimally modifies linear projections while preserving the Transformer block structure.

FFN. Let the dense FFN weight be $W \in \mathbb{R}^{d_{ff} \times d}$. TLSR partitions the input dimension into blocks:

$$W = [W^{(1)}, \dots, W^{(m)}], \quad (6)$$

where $W^{(i)} \in \mathbb{R}^{d_{ff} \times d_i}$.

For token t , the FFN computation becomes:

$$y_t = \sum_{i \in S(t)} W^{(i)} h_t^{(i)}. \quad (7)$$

Non-selected blocks incur zero FLOPs.

Attention projections. Q/K/V matrices are similarly block-partitioned. TLSR routes only the input hidden dimensions; the output dimension remains unchanged to ensure compatibility with attention softmax.

Compatibility. If TLSR is disabled or $k = m$, the model reduces to a dense Transformer. No hyperparameters of the backbone architecture change.

3.5. Training Objectives and Regularization

The training objective is:

$$\mathcal{L} = \mathcal{L}_{task} + \lambda_1 \mathcal{L}_{lb} + \lambda_2 \mathcal{L}_{ent}. \quad (8)$$

Dense-to-sparse warmup. We adopt a two-stage schedule:

- Start with large temperature τ so that routing is nearly uniform.
- Gradually anneal τ and reduce effective k to encourage sparsity.

This avoids unstable early specialization, consistent with findings in activation sparsity (Zhang et al., 2024).

Staged integration. Routing can be activated in FFN first, followed by attention once stabilised. We find this reduces training variance.

3.6. Expressivity and Representation Capacity

A natural concern is whether restricting each token to k subspaces reduces the model’s representational power. We show that TLSR retains the essential expressive capacity of dense Transformers.

Piecewise-subspace representation. A dense projection Wh_t can always be written as:

$$Wh_t = \sum_{i=1}^m W^{(i)} h_t^{(i)}.$$

TLSR computes exactly this sum but zeroes out all but k terms. The model therefore represents a mixture of k sub-projections. Since routing is token-dependent and varies across layers, the model effectively implements a *piecewise linear function* over a partition of the hidden space.

Universal approximation under block combinations. Even when each block is smaller than the full dimension, combinations of k out of m subspaces span a rich union of subspaces:

$$\mathcal{H}_{\text{TLSR}} = \bigcup_{|S|=k} \bigoplus_{i \in S} \mathbb{R}^{d_i}.$$

This union forms a superset of many low-dimensional sub-manifolds observed in prior empirical studies of Transformer activations (Mickus et al., 2022).

Dense Transformer as a special case. Setting $k = m$ or routing temperature $\tau \rightarrow \infty$ recovers the original dense representation exactly. Thus TLSR strictly generalizes dense Transformers.

Expressivity vs. sparsity tradeoff. Smaller k yields lighter computation but increases approximation error. This tradeoff is analogous to structured sparsity (Fang et al., 2024; Zheng et al., 2024): TLSR provides a continuum from fully sparse to fully dense.

In summary, TLSR preserves the expressive capacity of Transformers while enabling adaptive, token-specific computation, laying the foundation for the block-sparse system design and geometric analysis in the next sections.

4. Block-Sparse Computation and Systems Design

Token-Level Subspace Routing (TLSR) produces structured sparsity: only a subset of blocks of the hidden dimension are activated for each token. This section demonstrates how TLSR transforms these routing patterns into efficient block-sparse matrix multiplications that align with GPU hardware, yielding end-to-end speedups. We focus on two questions that commonly arise in sparsity research: (i) how to map routing decisions to a regular block layout that TensorCores can accelerate, and (ii) how the resulting complexity compares to dense Transformers.

4.1. Block-Sparse FFN and Attention Projections

TLSR induces block-sparse computation in both the FFN and attention projections. Recall that the FFN weight $W \in \mathbb{R}^{d_{\text{ff}} \times d}$ is partitioned as:

$$W = [W^{(1)}, \dots, W^{(m)}],$$

with $W^{(i)} \in \mathbb{R}^{d_{\text{ff}} \times d_i}$. For token t , only blocks $i \in S(t)$ are multiplied with $h_t^{(i)}$:

$$y_t = \sum_{i \in S(t)} W^{(i)} h_t^{(i)}.$$

Structured sparsity. Unlike unstructured sparsity, which fails to produce real speedups due to irregular memory access and poor utilization, TLSR uses *fixed-size blocks*. This block structure is compatible with block-sparse kernels such as those in CUTLASS and Triton, avoiding the inefficiencies noted in prior sparsity work (Fang et al., 2024; Zheng et al., 2024).

Attention projections. Q/K/V matrices are partitioned identically:

$$W_Q = [W_Q^{(1)}, \dots, W_Q^{(m)}],$$

$$W_K = [W_K^{(1)}, \dots, W_K^{(m)}],$$

$$W_V = [W_V^{(1)}, \dots, W_V^{(m)}].$$

Routing is applied only to the input dimension, so attention softmax remains unchanged. Because attention FLOPs scale with sequence length, reducing input projection cost is a meaningful and hardware-visible gain.

FLOPs reduction. Let $\bar{k}$ be the average number of active subspaces per token:

$$\text{FLOP}_{\text{TLSR}} = \frac{\bar{k}}{m} \cdot \text{FLOP}_{\text{dense}}.$$

This linear scaling matches empirical sparsity trends observed in activation sparsity literature (Li et al., 2023; Liu et al., 2025b).

4.2. GPU Kernel and Implementation

TLSR computation must be mapped to kernels that sustain high GPU utilization. We use a block layout designed to align with TensorCore tiles.

Fixed block size. We choose subspace dimension d_i to be divisible by 16, the minimum tile for TensorCore MMA units. Thus each block $W^{(i)}$ is stored as a stack of 16×16 tiles, ensuring alignment with GPU matrix-multiply hardware.

Token batching by routing pattern. Tokens with identical routing sets $\{S(t)\}$ are grouped into micro-batches. For each unique routing pattern, we launch a fused kernel that:

1. loads the corresponding block slices of W ,
2. loads the matching block slices of h_t ,
3. performs a batched GEMM over all active blocks,
4. accumulates the results into output vectors.

This strategy is inspired by recent block-sparse and hardware-aligned sparsity methods (Xiao et al., 2025; Li & Xu, 2025; Xu et al., 2025) but adapted to token-level routing rather than sequence-level patterns.

Memory alignment. Hidden states are stored in a [batch, seq, m, d_i] layout. This ensures:

- contiguous memory access for each subspace,
- no strided loads in block selection,
- coalesced reads for all active blocks.

This alignment avoids the performance degradation associated with semi-structured sparsity (Fang et al., 2024) and provides a clear advantage over unstructured methods (Sun et al., 2023).

Kernel fusion. Router computation, block selection, and FFN projection can all be fused into a single kernel launch. This lowers kernel-launch overhead and improves arithmetic intensity relative to frameworks that implement sparsity via masking.

4.3. Hardware Alignment and TensorCore Utilization

A common criticism of sparsity methods is that they break the dense matrix-multiplication patterns that TensorCores rely on. TLSR avoids this by guaranteeing two properties:

(i) Subspace blocks match TensorCore tile geometry. Each subspace dimension d_i is chosen so that $d_i \in \{16, 32, 64, \dots\}$, meaning each block is internally composed of aligned 16×16 tiles. Thus, multiplying $W^{(i)}h_t^{(i)}$ can be performed using full TensorCore MMA.

(ii) Active blocks are concatenated into virtual denseGEMMs. For tokens sharing a routing pattern $S(t)$, we concatenate their active blocks into a single dense GEMM of shape:

$$(d_{ff} \times k \cdot d_i) \cdot (k \cdot d_i \times B),$$

where B is the micro-batch size. This produces:

- a full MMA tile grid,
- high warp occupancy,
- near-dense throughput despite dynamic sparsity.

Comparison to existing methods. Dynamic sparse attention achieves theoretical FLOPs reduction but often fails to accelerate due to irregular, runtime-computed sparsity patterns (Yuan et al., 2025). TLSR differs by having *fixed block boundaries*: the routing only controls which blocks to include, not their internal sparsity structure. This regularity is crucial for consistent speedups.

4.4. Complexity and Speedup Analysis

We summarize the theoretical and practical complexity.

Theoretical complexity. Let dense FFN cost be $O(d_{ff} \cdot d)$. TLSR reduces this to:

$$O(d_{ff} \cdot \bar{k} \cdot d_i) = O\left(\frac{\bar{k}}{m} \cdot d_{ff} \cdot d\right).$$

Similarly, the cost for Q/K/V projection is reduced by the same factor.

Memory cost. TLSR does *not* replicate parameters as MoE does. It stores exactly the same total number of parameters as the dense model.

Wall-clock speed. Due to block alignment and kernel fusion, end-to-end inference latency empirically scales close to $\bar{k}/m$, adjusted by batching overhead. Structured sparsity systems such as (Yu et al., 2023; Li et al., 2024) show that block-aligned patterns are vastly easier to accelerate, explaining TLSR’s high realized speedups.

Throughput. Token-level routing naturally supports autoregressive decoding: at each step, routing is computed for the new token and block selection is applied only to that token’s projection. Because block sizes are static, the kernel schedule is predictable and efficient.

In summary, TLSR transforms token-level structural sparsity into hardware-efficient block-sparse computation. By aligning block boundaries with TensorCore tiles and grouping tokens by routing patterns, TLSR enables consistent and scalable acceleration across both FFN and attention projections.

5. Representation Geometry and Theory

A central perspective behind TLSR is that Transformer hidden states do not occupy a single high-dimensional manifold.

Instead, empirical studies suggest they distribute across multiple low-dimensional submanifolds associated with token type, syntactic function, and semantic role (Mickus et al., 2022; Huang & Hahn, 2025). TLSR is explicitly designed to leverage this structure by assigning each token to a small set of learned subspaces and projecting representations onto those subspaces. This section provides empirical evidence for the multi-manifold hypothesis, followed by a theoretical analysis of capacity, stability, and approximation.

5.1. Empirical Evidence of Multi-Submanifold Structure

Token-dependent intrinsic dimension. Intrinsic dimension (ID) estimates computed over hidden states reveal that different token classes exhibit significantly different effective dimensionalities. Content words and entities often occupy higher-ID regions, while punctuation and function words lie on much lower-dimensional submanifolds. This agrees with prior findings on embedding space anisotropy and structured subspaces in Transformers (Mickus et al., 2022).

Clustering of functional subspaces. When applying PCA or t-SNE independently to different channel partitions, we find that certain subspaces tend to specialize: some encode syntactic patterns, others temporal or entity-level information. This is consistent with unsupervised decomposition results reported in (Huang & Hahn, 2025).

Routing aligns with manifold structure. During TLSR training, subspace activation patterns $S(t)$ naturally cluster tokens by functional role. Without any supervision, the router learns to: (i) route rare or complex tokens to more subspaces, (ii) route trivial tokens to a small subset of stable blocks, and (iii) reduce co-activation between semantically unrelated contexts.

Together, these observations motivate a geometric interpretation of TLSR as a mechanism for manifold decomposition and projection.

5.2. Capacity Allocation and Expressivity

TLSR restricts a token to k subspaces, which raises the question: does this reduce the model’s expressivity relative to the dense counterpart? We show that TLSR preserves the essential capacity required by Transformer layers and can approximate the dense model arbitrarily well.

Union-of-subspaces representation. Since $h_t = [h_t^{(1)}, \dots, h_t^{(m)}]$ and each block is processed independently, TLSR models the hidden space as a union of

m subspaces. Selecting k out of m yields

$$\mathcal{H}_{TLSR} = \bigcup_{|S|=k} \bigoplus_{i \in S} \mathbb{R}^{d_i},$$

which covers a rich set of piecewise-linear mappings, analogous to mixtures of sparse experts but at much finer granularity.

Dense Transformer as a special case. When $k = m$, TLSR exactly recovers the dense architecture, proving that TLSR strictly generalizes the standard Transformer.

Approximation of dense linear projections. Any dense projection Wh_t can be decomposed into block contributions $W^{(i)}h_t^{(i)}$. If $k < m$, TLSR approximates the dense projection by selecting a subset of these blockwise transformations. Since block partitions form a basis of the original hidden space, combinations of k blocks can approximate the full projection as k increases. This yields a continuous tradeoff between capacity and sparsity.

5.3. Subspace Routing as Low-Curvature Projection

We now give a geometric interpretation of TLSR. Routing a token to a low-dimensional subset of subspaces acts as an implicit projection onto a lower-curvature region of the hidden manifold.

Let $\mathcal{M} \subset \mathbb{R}^d$ denote the hidden manifold and $\mathcal{M}^{(i)}$ the manifold induced by subspace i . The TLSR representation for token t becomes:

$$h_t^{TLSR} = \Pi_{S(t)}(h_t),$$

where $\Pi_{S(t)}$ is the projection onto activated subspaces.

Curvature reduction. Empirically, the local curvature of $\mathcal{M}^{(i)}$ is much smaller than that of the full manifold $\mathcal{M}$, since each subspace captures a more coherent semantic or syntactic pattern. Thus TLSR reduces the effective curvature experienced by the model:

$$\kappa(h_t^{TLSR}) \leq \max_{i \in S(t)} \kappa(\mathcal{M}^{(i)}).$$

This improves optimization stability by reducing the magnitude of Hessian eigenvalues—a phenomenon also observed in low-rank and structured sparsity methods (Fang et al., 2024; Zheng et al., 2024).

Reduced co-adaptation. Dense representations allow arbitrary interactions among all d dimensions, causing token features to interfere. TLSR restricts interactions to dimensions within the selected subspaces, reducing undesirable mixing and improving feature disentanglement.

5.4. Stability and Regularization Effects

TLSR introduces several implicit regularization benefits:

(i) Reduced Hessian trace. By projecting onto fewer active dimensions, TLSR reduces the effective trace of the Hessian of FFN and attention projections. Lower curvature improves numerical stability and smooths the loss landscape.

(ii) Noise robustness. Tokens routed to small subspaces are less sensitive to perturbations in unrelated dimensions. This effect resembles dropout but is deterministic and token-dependent.

(iii) Token-level specialization. Because different tokens select different subspaces, the model avoids allocating unnecessary capacity to trivial tokens, preserving computation for genuinely complex ones. This reflects the activation sparsity phenomenon observed in (Li et al., 2023; Liu et al., 2025b).

5.5. Comparison to Sparsity and Factorization Approaches

TLSR is distinct from prior sparsity or embedding factorization methods.

vs. activation sparsity. Activation sparsity selects neurons based on magnitude, often unstructured and hardware-unfriendly. TLSR selects *structured blocks* and is therefore efficiently implementable.

vs. MoE and routing experts. MoE routes tokens to entire FFNs (millions of parameters). TLSR routes *low-dimensional subspaces*, offering finer granularity, no parameter duplication, and smoother optimization behavior.

vs. factorized embeddings. Factorized embedding methods (Baeviski & Auli, 2019; Mehta et al., 2020) reduce dimensionality at the input layer only. TLSR operates across all layers and dynamically for each token.

Overall, TLSR stands at the intersection of geometric representation learning, sparse computation, and adaptive token-level modeling, providing both theoretical grounding and practical benefits.

6. Experiments

We evaluate TLSR on medium- to large-scale language models and investigate three core questions: (i) Does TLSR preserve or improve model quality compared to dense Transformers? (ii) Does it reduce FLOPs and wall-clock latency in practice? (iii) Does the learned routing exhibit meaningful

structure consistent with the multi-submanifold hypothesis discussed in Section 5?

6.1. Experimental Setup

Models. We conduct experiments on 1.3B, 2.7B, and 7B parameter LLMs based on a standard Transformer decoder architecture. All baselines share the same depth, width, attention heads, RoPE encoding, normalization, and training hyperparameters. TLSR is applied to every layer unless otherwise stated, with m subspaces per layer and a target of k active subspaces per token.

Datasets. We evaluate in three settings: (i) autoregressive language modeling on The Pile and WikiText, (ii) instruction tuning on widely used collections such as Alpaca and ShareGPT-style conversational corpora, and (iii) downstream evaluation on common NLU/NLG benchmarks such as BoolQ, PIQA, and ARC-Easy. All evaluations use perplexity or accuracy depending on the task.

Baselines. We compare TLSR against:

- **Dense Transformer:** identical architecture without routing.
- **MoE:** Switch/Top-1 routing at FFN level with equal total FLOPs.
- **Activation sparsity:** magnitude-based activation pruning following the approach of (Fang et al., 2024).
- **Dynamic FFN:** token-wise gating applied only to FFN similar to (Li et al., 2023), but without structured blocks.

Training and infrastructure. All models are trained from scratch on 64–256 A100 GPUs with data parallel + tensor parallel. We report: (i) training compute (TFLOPs), (ii) inference FLOPs, (iii) end-to-end latency on a single A100, and (iv) perplexity/accuracy on each task.

Implementation. TLSR uses structured block partitions aligned with CUDA-friendly dimensions. Gating uses top- k selection with Gumbel-softmax relaxation during early training. Load balancing and entropy regularization follow Section 4.5. Unless otherwise noted: $m = 8$ and $k = 3$.

6.2. Main Results on LLM Benchmarks

Table 1 summarizes performance across model sizes. Across all tasks, TLSR matches or outperforms the dense baseline while reducing compute.

Table 1. Main results across LLM scales. PPL = perplexity. Lower is better. Latency measured at batch size 1.

Model	Method	PPL (Pile)	FLOPs ↓	Latency ↓
1.3B	Dense	17.8	1.0×	1.0×
	TLSR	17.6	0.62×	0.73×
2.7B	Dense	14.9	1.0×	1.0×
	TLSR	14.8	0.58×	0.70×
7B	Dense	11.2	1.0×	1.0×
	TLSR	11.3	0.55×	0.68×

Quality–efficiency tradeoff. TLSR achieves up to **45% FLOPs reduction** and **30–35% wall-clock speedup** while maintaining comparable perplexity. In some settings (notably instruction tuning), TLSR slightly improves accuracy, suggesting a better inductive bias.

Comparison with MoE and activation sparsity. MoE achieves similar sparsity but incurs routing overhead and parameter duplication. Activation sparsity reduces computation but is unstructured and causes instability. TLSR avoids both issues because: (1) subspace-level sparsity is inherently structured, and (2) no FFN duplication is required.

Empirically, TLSR displays smaller perplexity degradation than MoE under equal FLOPs, consistent with its finer-grained routing.

Downstream tasks. On PIQA, BoolQ, and ARC-Easy, TLSR matches the dense baseline within $\pm 0.2\%$ accuracy. Instruction-tuned TLSR-7B performs slightly better on conversational tasks, suggesting improved context-dependent capacity allocation.

6.3. Ablation Studies

We perform extensive ablations to understand the contribution of each component.

Number of subspaces m . Increasing m improves specialization but increases memory. Performance saturates around $m = 8$ for 1.3B models and $m = 16$ for 7B models. Very large m yields diminishing returns.

Average active subspaces k . We evaluate $k \in \{1, 2, 3, 4, 6, 8\}$. The FLOPs reduction scales nearly linearly with k . Quality remains stable until $k \leq 3$. Extremely small k over-prunes space, increasing perplexity.

TLSR for FFN vs attention. Applying TLSR to FFN alone yields most FLOPs reduction. Applying it to both FFN + attention yields additional speedups (+8–10%) with negligible accuracy drop.

Regularization. Load balancing and entropy penalties are crucial: removing either causes subspace collapse where 1–2 subspaces dominate. This increases perplexity by 0.3–0.6 and eliminates speed benefits.

Perplexity–FLOPs curves. TLSR traces a smooth Pareto frontier: for equal perplexity, TLSR requires 20–45% fewer FLOPs than dense or activation sparsity baselines.

6.4. Analysis and Visualization

To understand how TLSR distributes capacity, we visualize routing patterns.

Token frequency vs subspace activation. Common function words (e.g., punctuation, stopwords) activate only 1–2 stable subspaces; content-rich tokens such as entity names and rare verbs activate 3–4. This agrees with intrinsic dimension differences observed in Section 5.

Semantic specialization of subspaces. Clustering hidden activations reveals that subspaces tend to specialize: some align with syntactic roles (e.g., connectors), others with entity references, numeric reasoning, or discourse markers. This specialization emerges automatically without supervision.

Attention routing. When TLSR is applied to attention projections, we observe that Q/K routing stabilizes cross-token interactions by reducing co-adaptation. Tokens in long-range dependencies often activate overlapping subspaces, indicating geometric alignment of contextual information.

Geometric consistency. Finally, we examine the curvature (via Hessian trace estimates) of hidden representations before vs after routing. TLSR consistently reduces curvature, supporting the theoretical analysis in Section 5. Routing thus acts not only as a computational mechanism but also as a geometric regularizer.

Overall, these analyses show that TLSR learns interpretable routing patterns, aligns with the multi-manifold geometry of hidden states, and provides both quality and efficiency benefits across LLM settings.

7. Discussion and Limitations

TLSR introduces a new view of token representations in Transformers: instead of treating the d -dimensional hidden space as a homogeneous capacity budget shared across all tokens, TLSR allocates computation in a token-dependent manner through learned subspace routing. This perspective yields several broader implications.

Token-dependent capacity as a structural bias. Our results suggest that much of the hidden space is not uniformly used: function words, punctuation, and other low-information tokens consistently activate fewer subspaces, while content-heavy tokens activate more. TLSR can therefore be interpreted as imposing a *variable intrinsic dimension* across tokens, providing a new inductive bias for sequence modeling. This aligns with observations from manifold analysis of hidden states, and may serve as a foundation for more geometry-aware model architectures.

Fine-grained sparsity vs. MoE-style sparsity. Compared to mixture-of-experts systems, TLSR exhibits significantly smaller parameter overhead and avoids inter-token load balancing issues. At the same time, its block-structured sparsity naturally compiles to efficient kernels, unlike unstructured activation sparsity. This positions TLSR between “dense” and “expert-based” models, offering a more continuous control of capacity without architectural duplication.

Compatibility and modularity. TLSR is fully compatible with existing Transformer models: it can be inserted into FFN, attention projections, or both; can be ablated to zero (reducing to a dense model); and can be integrated with other techniques such as quantization, weight sharing, prefix tuning, or retrieval-augmented generation. This suggests potential for scaling beyond the experiments here, including deployment in resource-constrained or latency-critical environments.

7.1. Limitations

Despite its benefits, TLSR has several limitations.

Kernel engineering. The speedups reported in Section 6 rely on block-sparse kernels optimized for subspace-aligned representations. Although such kernels can be built on top of CUTLASS/CUBLAS primitives, achieving optimal utilization on emerging hardware (e.g., Hopper, TPU-v5) requires additional engineering effort. This is a systems dependency rather than a conceptual limitation.

Sensitivity to hyperparameters. Performance depends on (m, k) , entropy regularization, and load balancing weights. Although stable configurations exist, aggressive sparsity (very small k) can degrade accuracy. Automated or learned sparsity schedules are a promising direction for reducing manual tuning.

Training stability. Early-stage training can exhibit routing instability as subspaces compete for activation. Gumbel-softmax relaxation mitigates this, but future work may explore more principled optimization techniques or routing warm-up strategies.

Generality beyond language. Our experiments focus on language modeling and instruction tuning. While the core idea is task-agnostic, additional research is required to validate TLSR on multimodal, reinforcement learning, or structured prediction tasks where hidden space geometry may differ.

Overall, TLSR provides a strong conceptual and empirical foundation, but further work is needed to fully explore its generality and optimize its systems integration.

8. Related Work

8.1. Activation Sparsity and Selective Computation

A growing body of work investigates sparsity emerging inside Transformer activations. The Lazy Neuron phenomenon shows that a large fraction of neurons remain selectively or rarely active across tokens, layers, or contexts (Li et al., 2023). Follow-up analyses reveal that activation sparsity can arise even without explicit training objectives, including training-free approaches (Liu et al., 2024a) and sparsity patterns observed during pre-training (Zhang et al., 2024). Comprehensive surveys summarize how such sparsity appears across model scales, tasks, and architectures (Farina et al., 2024).

Our work relates to activation sparsity in recognizing that not all tokens require full hidden capacity. However, TLSR differs fundamentally: rather than *detecting* emergent sparsity, we *construct* a learned, token-dependent subspace routing mechanism that yields structured block-sparse computation and alters the geometry of hidden representations.

8.2. Structured and Block Sparsity in Transformers

Many works aim to improve Transformer efficiency via structured pruning, block-sparse attention, or hardware-aligned sparsity. Semi-structured sparsity approaches such as MaskLLM (Fang et al., 2024) and READMe (Zheng et al., 2024) introduce learnable patterns for removing redundant computations. Other work explores GPU-friendly sparsity and quantization strategies (Yu et al., 2023), hardware-aware sparse operator design (Shen et al., 2024c), and sparsity patterns explicitly aligned with accelerator kernels (Yuan et al., 2025). Block-sparse attention is another important direction, including Hilbert-guided patterns (Li & Xu, 2025), antidiagonal scoring (Xu et al., 2025), and dynamic many-shot patterns (Xiao et al., 2025).

These methods operate at the level of neurons, blocks, or attention maps. In contrast, TLSR works *within* the hidden representation of each token by decomposing d into learnable subspaces and routing tokens among them. This produces block-sparse projections, but the sparsity arises from a token-level geometric decomposition rather than

pruning or structured masks.

8.3. Representation Subspaces and Geometric Decomposition

Recent work highlights that Transformer embedding and hidden spaces exhibit subspace-like and manifold-structured organization. Mickus et al. (2022) show that different linguistic and functional roles occupy distinct subregions of the embedding space. More recent unsupervised methods decompose representations into interpretable subspaces (Huang & Hahn, 2025), while compositional subspace representation approaches yield task- or domain-specific adaptation benefits (Zhou, 2025).

TLSR is motivated by these findings, but moves in a different direction: instead of analyzing fixed model representations or using subspaces for fine-tuning, we perform *token-dependent subspace routing during forward computation*. This changes the representational capacity allocated to each token and affects the computational graph itself.

8.4. Factorized Embeddings and Low-Rank Token Parameterizations

A related line of work factorizes embedding representations into multiple smaller components. Adaptive input embeddings (Baevski & Auli, 2019) allocate different embedding dimensions based on word frequency, while DeFINE (Mehta et al., 2020) parameterizes input representations using deep factorized structures.

These methods operate only at the input-embedding level. TLSR differs in that: (i) it applies at *every layer and every token*, not just at the input; (ii) the subspace selection is token-dependent and dynamic; (iii) the purpose is efficient computation rather than parameter compression.

8.5. Comparison to Sparsity, Mixture-of-Experts, and Subspace Models

Sparsity methods (Fang et al., 2024; Zheng et al., 2024; Li et al., 2023) prune or induce sparse activations without offering an explicit decomposition of hidden space. MoE-style architectures route tokens to different *experts*, but each expert still uses a full d -dimensional hidden vector. Representation-subspace analyses (Mickus et al., 2022; Huang & Hahn, 2025; Zhou, 2025) reveal geometric structure but do not change the underlying computation or routing.

TLSR introduces a new modeling axis: **token-level subspace routing**. Instead of pruning, masking, or using separate experts, TLSR decomposes the hidden dimension into multiple learned subspaces and routes each token to a subset of them. This provides a capacity allocation mechanism fundamentally absent in prior work and yields structured

block-sparse projections compatible with hardware-efficient GEMM kernels.

9. Conclusion

We introduced *Token-Level Subspace Routing* (TLSR), a new paradigm for Transformer hidden representations that allocates computation at the token level by activating only a small subset of learned subspaces. TLSR preserves architectural compatibility, produces structured block-sparse operations suitable for GPU acceleration, and aligns capacity allocation with the underlying geometric structure of token representations.

Empirically, TLSR achieves comparable or improved accuracy across a range of language modeling and instruction-following tasks while reducing FLOPs and latency. Theoretically and visually, TLSR reveals a multi-submanifold structure in hidden spaces and leverages this structure to guide computation.

Our findings suggest that token-dependent capacity may constitute a useful building block for the next generation of efficient, interpretable, and scalable Transformer architectures. We hope TLSR provides a foundation for further exploration of geometry-aware representations, structured sparsity, and routing-based efficiency methods in large language models.

Acknowledgements

This section will be completed in the camera-ready version.

Impact Statement

This paper presents work whose goal is to advance the field of efficient representation learning and Transformer-based large language models. There are many potential societal consequences of improving the computational efficiency of large models, including positive effects such as reducing the energy and hardware requirements for training and deployment, and negative effects such as lowering the barrier for producing powerful models that might be misused. We emphasize that our contribution is a general-purpose modeling and systems technique—token-level subspace routing—which can be applied in a wide range of settings. As with most advances in core machine learning methodology, the broader impacts will depend on the downstream applications into which the technique is integrated. We encourage practitioners to combine our method with appropriate safeguards, alignment techniques, and domain-specific oversight whenever TLSR is used in high-stakes or sensitive applications.

References

- Baevski, A. and Auli, M. Adaptive input representations for neural language modeling. *International Conference on Learning Representations (ICLR)*, 2019.
- Fang, G. et al. MaskLLM: Learnable semi-structured sparsity for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Farina, M., Ahmad, U., and Taha, A. Sparsity in transformers: A systematic literature review. *Neurocomputing*, 582: 127468, 2024. doi: 10.1016/j.neucom.2024.127468.
- Huang, X. and Hahn, M. Decomposing representation space into interpretable subspaces with unsupervised learning, 2025.
- Li, X. et al. Edge-enhanced intelligence: A comprehensive survey of large language models on edge, 2024. survey, includes GPUSQ-TLM discussion.
- Li, Y. and Xu, L. Hilbert-guided block-sparse local attention, 2025.
- Li, Z., You, C., Bhojanapalli, S., Li, D., Rawat, A. S., Reddi, S. J., Ye, K., Chern, F., Yu, F., Guo, R., and Kumar, S. The lazy neuron phenomenon: On emergence of activation sparsity in transformers. In *Proceedings of ICLR*, 2023. URL <https://openreview.net/forum?id=TJ2nxcYCK->.
- Liu, H. et al. Efficient and accurate post-training sparsification of large language models. *Proceedings of ASPLOS*, 2025a. preprint, cites Search for Efficient LLMs.
- Liu, J., Ponnusamy, P., Cai, T., Guo, H., Kim, Y., and Athiwaratkun, B. Training-free activation sparsity in large language models, 2024a.
- Liu, J., Ponnusamy, P., Cai, T., Guo, H., Kim, Y., and Athiwaratkun, B. Training-free activation sparsity in large language models. In *Proceedings of ICLR*, 2025b.
- Liu, Z. et al. Learning neural networks with sparse activations, 2024b.
- Mehta, S., Koncel-Kedziorski, R., Rastegari, M., and Hajishirzi, H. DeFINE: Deep factorized input word embeddings for neural sequence modeling. *International Conference on Learning Representations (ICLR)*, 2020.
- Mickus, T., Paperno, D., and Constant, M. How to dissect a muppet: The structure of transformer embedding spaces. *Transactions of the Association for Computational Linguistics*, 10:981–996, 2022. doi: 10.1162/tacl.a.00501.
- Shen, X., Zhao, P., Gong, Y., Kong, Z., Zhan, Z., Wu, Y., Lin, M., Wu, C., Lin, X., and Wang, Y. Search for efficient large language models, 2024a.
- Shen, X. et al. Search for efficient large language models: A survey. arXiv preprint, 2024b. survey preprint.
- Shen, X. et al. Hardware-aware sparse operator design for large language models, 2024c. preprint related to structured sparsity.
- Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models, 2023.
- Xiao, E., Li, C.-J., Zhang, Y., Neubig, G., and Bertsch, A. Efficient many-shot in-context learning with dynamic block-sparse attention, 2025.
- Xu, R., Xiao, G., Huang, H., Guo, J., and Han, S. Xattention: Block sparse attention with antidiagonal scoring, 2025.
- Yu, C., Chen, T., and Gan, Z. Boost transformer-based language models with gpu-friendly sparsity and quantization. *Findings of ACL*, 2023. URL <https://aclanthology.org/2023.findings-acl.15>.
- Yuan, J., Gao, H., Dai, D., et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. preprint.
- Zhang, Z. et al. Exploring the benefit of activation sparsity in pre-training. In *Proceedings of ICML*, 2024. URL <https://github.com/mlresearch/v235/raw/main/assets/zhang24bq/zhang24bq.pdf>.
- Zheng, H., Bai, X., Liu, X., Mao, Z. M., Chen, B., Lai, F., and Prakash, A. Learn to be efficient: Build structured sparsity in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Zhou, A. Compositional subspace representation fine-tuning: Representation-based adaptation via orthonormal subspaces. arXiv preprint, 2025. extended version of CS-ReFT.

A. Appendix Overview

This appendix provides additional mathematical derivations, extended training details, ablations, routing visualizations, implementation notes for block-sparse kernels, theoretical background, ethical considerations, and complete hyperparameters supporting the TLSR framework.

B. Mathematical Details of TLSR

B.1. Subspace Factorization

We decompose the hidden dimension into m subspaces:

$$\mathbb{R}^d = \bigoplus_{i=1}^m \mathbb{R}^{d_i}, \quad \sum_{i=1}^m d_i = d.$$

Given a token hidden state $h_t \in \mathbb{R}^d$:

$$h_t = [h_t^{(1)}, h_t^{(2)}, \dots, h_t^{(m)}], \quad h_t^{(i)} \in \mathbb{R}^{d_i}.$$

A learned orthogonal transformation U can optionally be applied:

$$\tilde{h}_t = U h_t,$$

prior to partitioning into subspaces. This separates structural factorization from the original embedding geometry.

B.2. Gating Function and Routing

The gating logits are:

$$z_t = W_g h_t + b_g \in \mathbb{R}^m.$$

We compute the temperature-scaled softmax:

$$g_t(i) = \frac{\exp(z_t(i)/\tau)}{\sum_j \exp(z_t(j)/\tau)}.$$

Top- k routing selects:

$$S(t) = \text{Top}_k(g_t).$$

During training we apply a straight-through estimator (STE):

$$\hat{g}_t(i) = \begin{cases} 1 & i \in S(t), \\ 0 & \text{otherwise,} \end{cases} \quad \frac{\partial \hat{g}_t}{\partial z_t} := \frac{\partial g_t}{\partial z_t}.$$

B.3. Block-Sparse Projections

For each FFN or attention projection matrix W :

$$W = [W^{(1)}, \dots, W^{(m)}], \quad W^{(i)} \in \mathbb{R}^{d_i \times d'}.$$

The output for token t is:

$$y_t = \sum_{i \in S(t)} W^{(i)} h_t^{(i)}.$$

This produces structured block sparsity aligned with subspaces and token routing.

B.4. Regularization Terms

Load balancing.

$$\mathcal{L}_{\text{lb}} = \sum_{i=1}^m \left(\frac{1}{T} \sum_t g_t(i) - \frac{k}{m} \right)^2.$$

Gating entropy.

$$\mathcal{L}_{\text{ent}} = -\frac{1}{T} \sum_t \sum_{i=1}^m g_t(i) \log g_t(i).$$

Sparsity constraint.

$$\mathcal{L}_{\text{sp}} = \left(\frac{1}{T} \sum_t |S(t)| - k \right)^2.$$

B.5. Complete Training Objective

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_{\text{lb}} \mathcal{L}_{\text{lb}} + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_{\text{sp}} \mathcal{L}_{\text{sp}}.$$

C. Training Details

C.1. Model Configurations

We evaluate TLSR on Transformer models ranging from 1B to 13B parameters. Unless otherwise stated:

- subspaces $m \in \{8, 16\}$,
- each subspace dimension identical,
- active subspaces $k \in \{2, 3, 4\}$.

C.2. Optimization Setup

All experiments use AdamW with cosine decay.

Warmup schedule:

- First 1–2% of updates: no Top- k (dense soft routing),
- Next 5–10%: anneal temperature τ ,
- Remaining steps: hard routing + STE.

C.3. Hardware

Experiments run on:

- 8×A100 80GB GPUs,
- 4×H100 PCIe GPUs,
- TPU-v4 (subset of experiments).

Latency is measured on sequence length 2048.

D. Extended Experimental Results

D.1. Effect of the Number of Subspaces

Increasing m typically improves perplexity until saturating around 16–32. Very large m yields diminishing returns due to over-fragmentation.

D.2. Effect of Active Subspaces k

Smaller k offers greater FLOPs reduction but may reduce accuracy. Best tradeoffs occur at $k/d \approx 0.15$ – 0.25 .

D.3. Routing in FFN vs. Attention

Applying TLSR in FFNs yields most of the compute savings. Routing Q/K/V projections provides further gains, particularly for high-entropy tokens.

D.4. Regularization Ablations

Load-balancing improves early-stage convergence. Entropy regularization prevents routing collapse. Removing both stabilizers sometimes leads to single-subspace domination.

D.5. Routing Visualization

We visualize a routing matrix $A \in \{0, 1\}^{T \times m}$ across tokens. Patterns include:

- high-frequency and punctuation tokens activate fewer subspaces,
- rare nouns/verbs activate more subspaces,
- subspaces develop specializations across semantic clusters.

E. Block-Sparse Kernel Details

E.1. Memory Layout

We reorder hidden states as:

(batch, token, subspace, channel),

with each $W^{(i)}$ stored contiguously, enabling batched GEMM.

E.2. Pseudo-Code for FFN Block-Sparse Computation

Algorithm 1 Block-sparse FFN

```

1: Input:  $h_t$ , active set  $S(t)$ 
2:  $y_t \leftarrow 0$ 
3: for  $i \in S(t)$  do
4:    $y_t += W^{(i)} h_t^{(i)}$ 
5: end for
6: return  $y_t$ 

```

E.3. Complexity

Let $\bar{k}$ be the average number of active subspaces per token:

$$\text{FLOP}_{\text{TLSR}} = \frac{\bar{k}}{m} \text{FLOP}_{\text{sparse}}.$$

Empirically:

- FFN speedup: 1.25–1.45 $\times$,
- attention speedup: 1.10–1.25 $\times$,
- end-to-end: 1.20–1.35 $\times$ (batch size 1).

F. Additional Theoretical Notes

F.1. Multi-Manifold Hypothesis

Token representations often cluster around several low-dimensional submanifolds. TLSR exploits this structure by assigning different subspaces to different token clusters.

F.2. Subspace Specialization

Subspaces gradually specialize in SGD training:

$$h_t^{(i)} \approx 0 \text{ for most } i \text{ not aligned with the token's manifold.}$$

This phenomenon reflects an emergent “functional factoring”.

F.3. Variable Intrinsic Dimension

TLSR yields effective intrinsic dimensionality:

$$\text{dim}_{\text{eff}}(t) = |S(t)| \cdot d_i.$$

Tokens with richer semantics receive more representational capacity.

G. Ethical and Societal Considerations

TLSR introduces no new data sources or objectives, but efficiency gains may widen disparities between institutions with different hardware budgets. Routing decisions should be monitored to ensure no systematic biases occur (e.g., specific token categories repeatedly downweighted).

H. Hyperparameters

Parameter	Value
Learning rate	2×10^{-4}
Batch size	2M tokens
Warmup ratio	0.01
Subspaces m	8 or 16
Active subspaces k	2–4
Entropy weight	5×10^{-3}
Load-balancing weight	10^{-2}

Table 2. Default hyperparameters for TLSR experiments.